This Is An Old Thread!

You are viewing and about to reply to a Thread that is 60 or days older. Please refrain from bumping threads this old!

TUTORIAL Editing the .CAM file through hex

HardRain

TUTORIAL Editing the .CAM file through hex 28 Jul 2021 at 20:41 JADERLINK, Biohazard4X, and 1 more like this

Post by HardRain on 28 Jul 2021 at 20:41

Hi there, guys.

Recently I've been studying and testing the .cam file a lot and got enough knowledge to make it work flawlessly, now we can edit it through HxD to add some static cameras & create new entries, so I decided to share with all of you what I've learned.
Huge thanks to Biohazard4X for helping me a lot in this.

This tutorial will be very lenghty and I will update it eventually, it will be divided into 4 parts:

- Understanding and copying the data
- More in-depth details and fixing errors
- Adding a new camera into our custom CAM
- Inserting new camera into a original CAM

Tools you will need:

Crzosk's Cam Tool
RE4 Level Viewer
HxD

ONE ADVICE: If your PC is powerful enough to run 3DS MAX software, I'd highly suggest

NEW MEMBER
NEW MEMBER
JUNIOR MEMBER
FULL MEMBER
SENIOR MEMBER
DEDICATED MEMBER
MODDING INFECTED
MODDER
MODERATOR
CFX DESIGNER
ADMIN

Retired for a while
Posts: 184♂
Member is Online

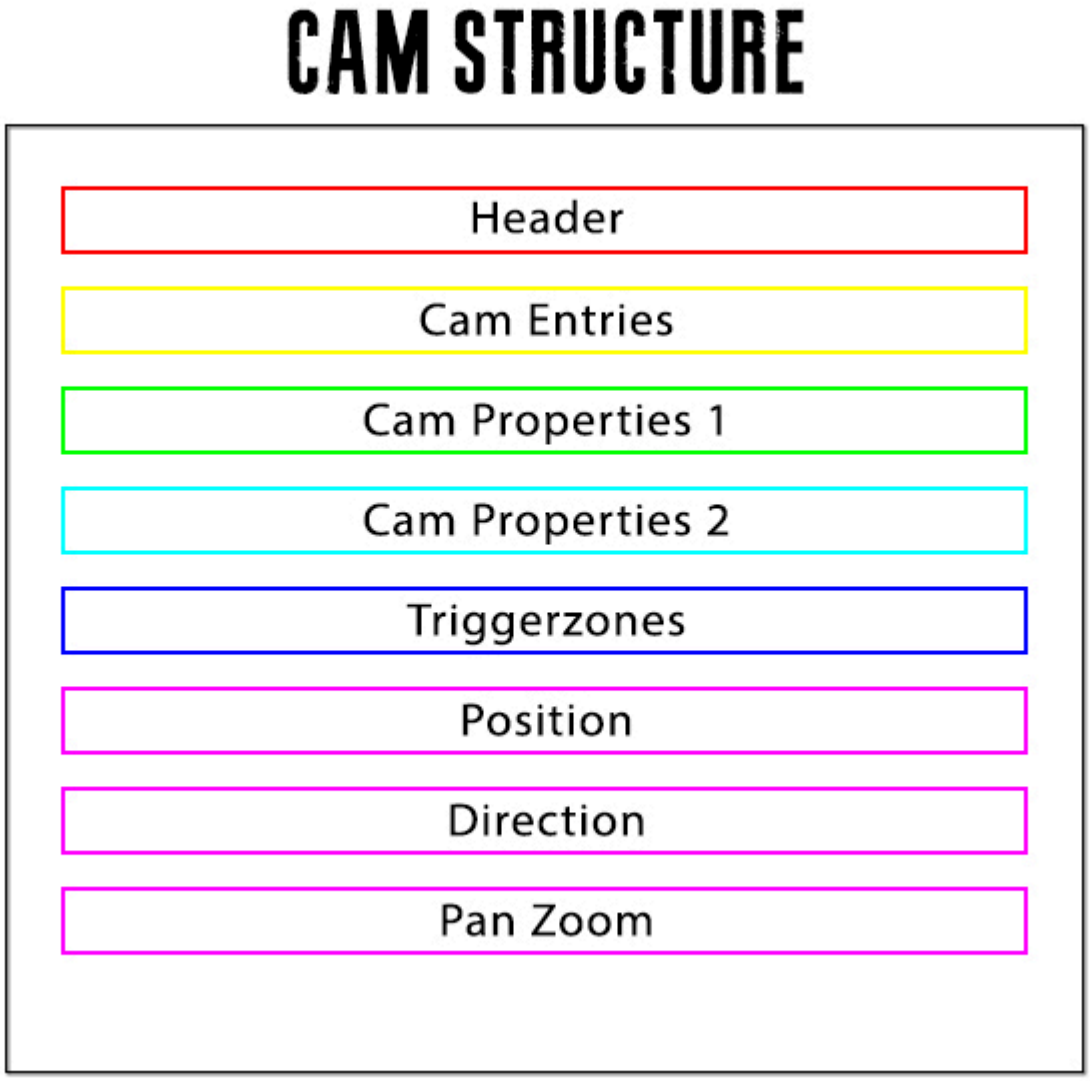
you to use it with MarioKart64n Script, because you can do these steps with just a few clicks. This tutorial is for those who doesn't have a nice PC, 3DS MAX or for those who always likes to learn something new, like me.
-- --

UNDERSTANDING AND COPYING THE DATA

PART 01

*Please make a backup of your original files before any edits

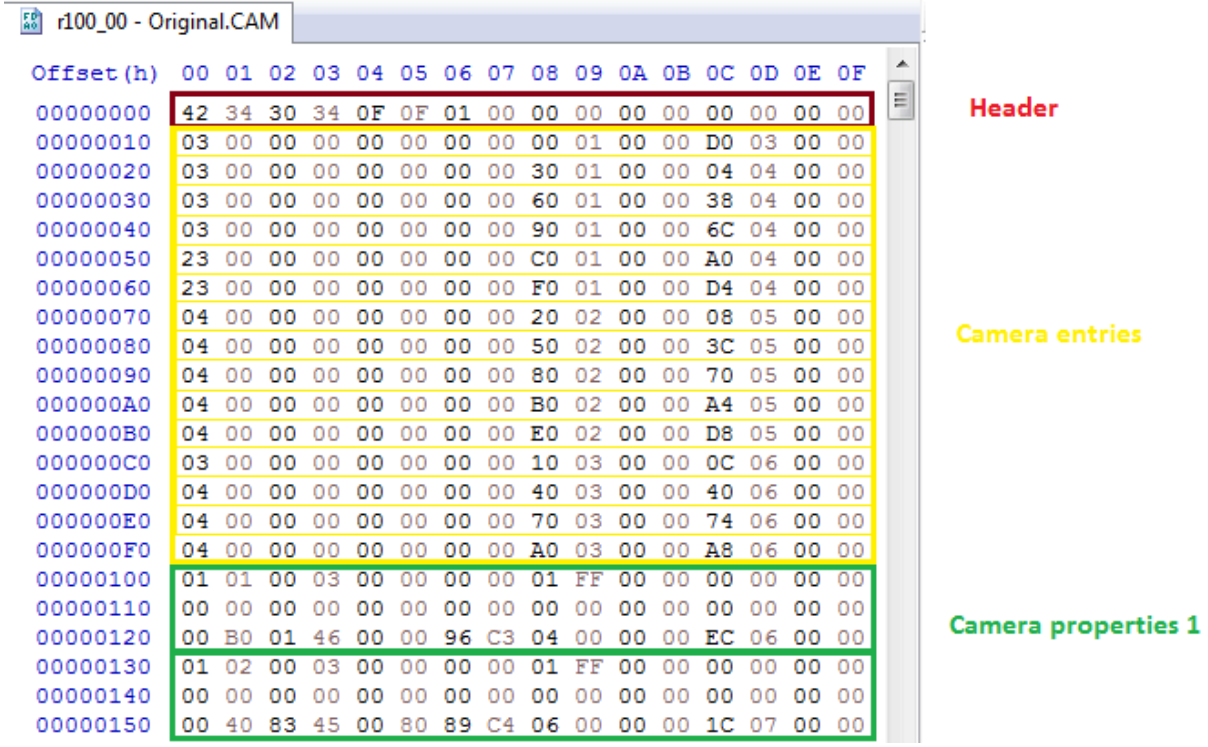
First things first, you need to know how the file structure is built to identify the bytes correctly, take a look at this:



CAM Structure

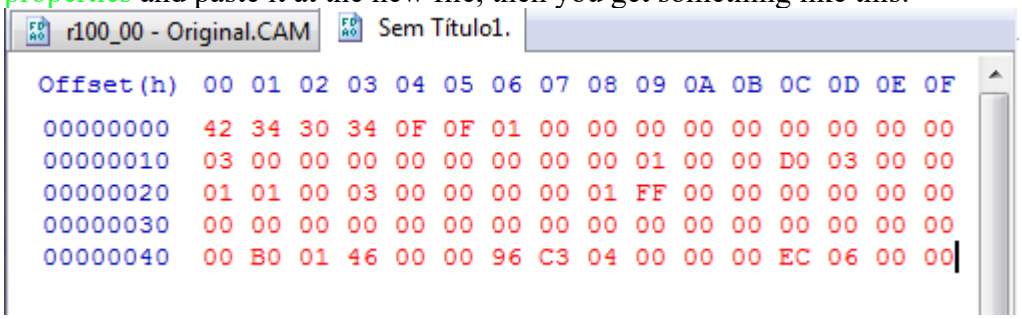
Header: Contains Index bytes
Cam Entries: Contains triggerzone type, pointers for offsets
Cam Properties 1: Contains LIT Group, triggerzone type, lower limit, higher limit, pointers
Cam Properties 2: Contains Index, camera type, data set, a lot of pointers
Triggerzones: Contains triggerzones in X,Y and Z
Position: Contains camera position
Direction: Contains camera direction (seems like rotation)
PAN Zoom: Contains Field of View

With that being said, I will be using the first room camera for this tutorial, so let's open r100_00.CAM at HxD and analyze it.



Here we can see the start of the structure, with this information we can start creating a new camera from scratch, so press **CTRL + N** to create a new file on HxD.

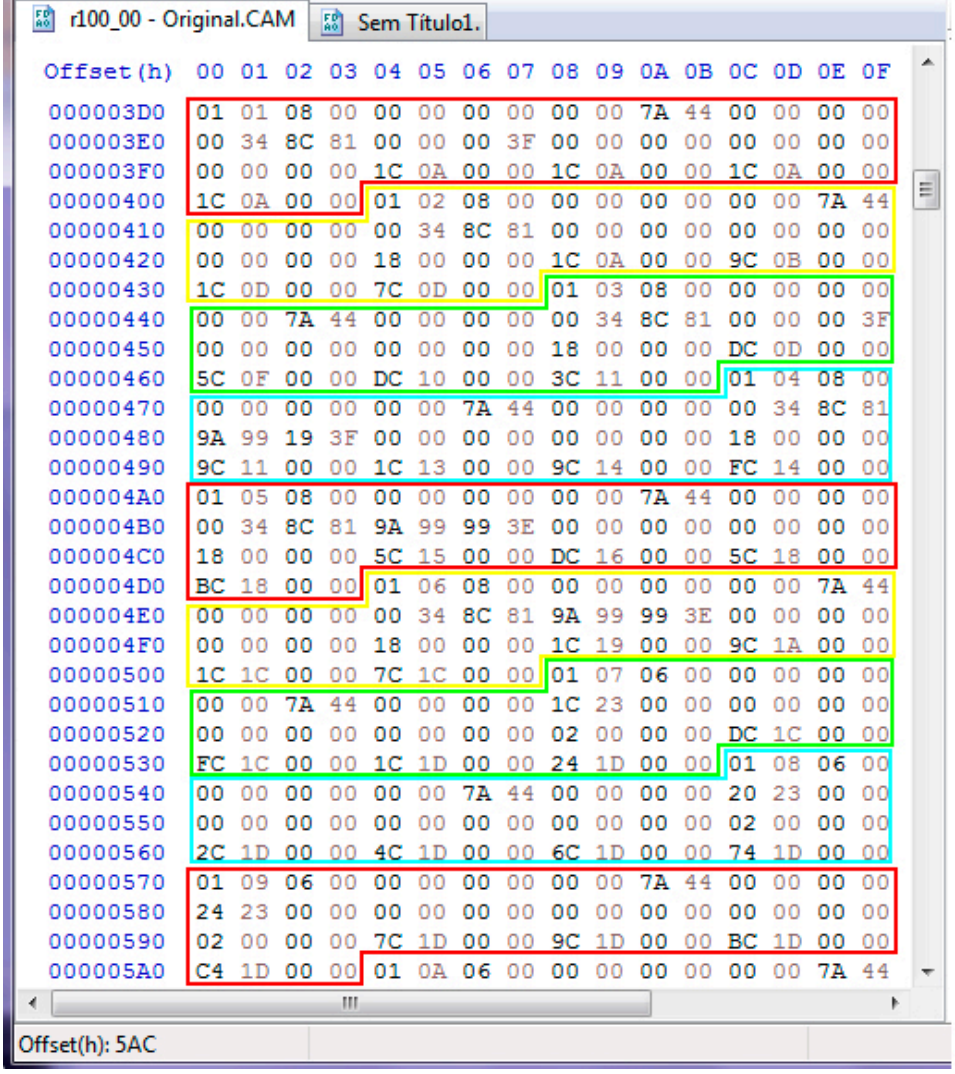
After creating the new file, return to the r100_00.CAM tab (you can alternate tabs by pressing **CTRL + TAB**), copy the header, the very first camera entry and the very first camera properties and paste it at the new file, then you get something like this:



Now let's return to r100_00.CAM to find the rest of the structure.

If you look at the index bytes of the header, you can see a 0F 0F, that value means we have a total of 15 cameras in r100_00.CAM, so with this information you can notice there are 15 entries and, consequently, there will be 15 camera properties.

Scroll down by counting the indexes or simply go to offset **3D0** to get to the next structure part, the camera properties 2:



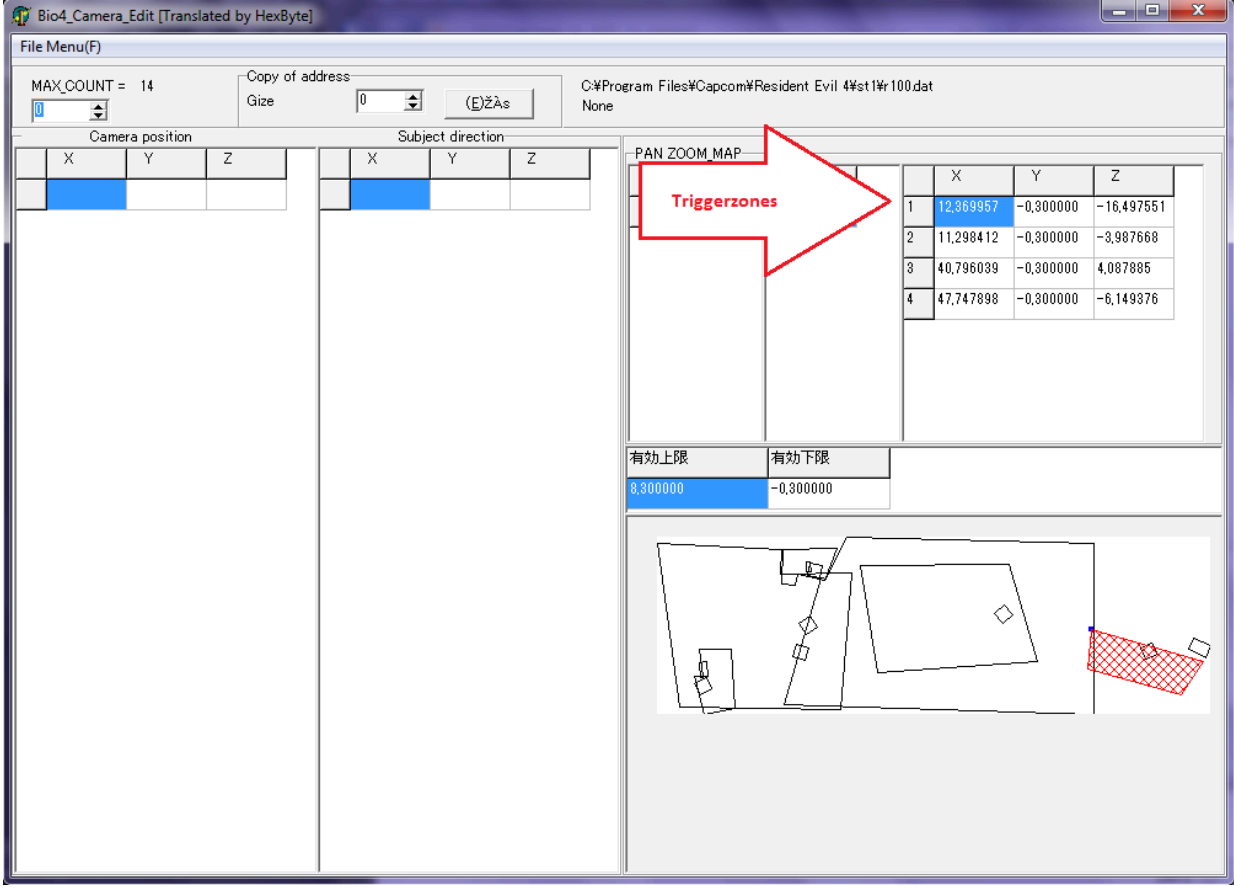
Camera properties 2

Each camera properties 2 have a lenght of 34 bytes, so let's copy the very first one and paste it below at our custom camera. I will explain everything afterwards, do not worry.

After pasting, now we need the last parts of the structure: triggerzones, position, direction and pan zoom, so let's move.
You can scroll down and go counting the camera properties indexes or simply go to offset **6E0**.

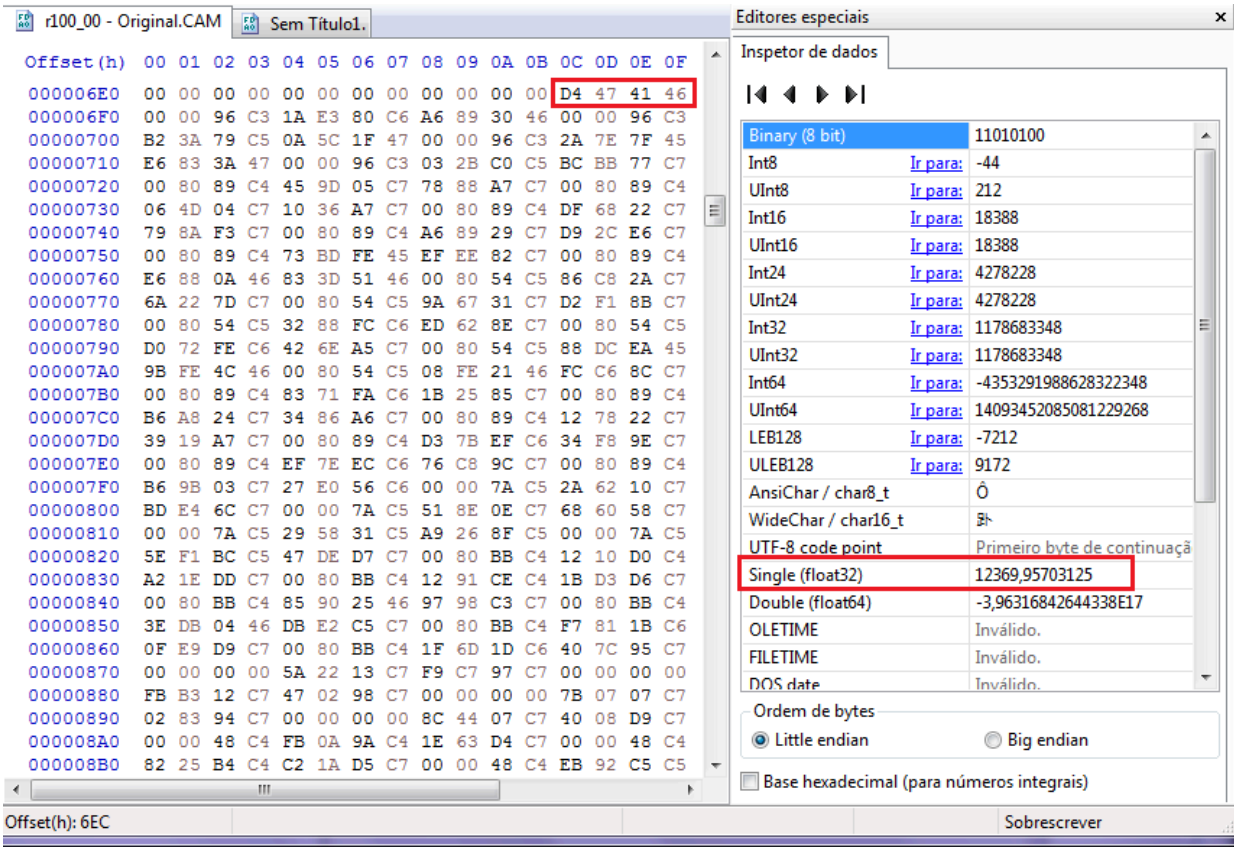
Now things will get a bit more complicated, we are in the triggerzone area.
Every triggerzone has a X, Y and Z value, consistent in float values (4 bytes for each X, Y and Z), and the triggerzone can contain 4 or 6 vertexes in total.

To make this easier we can use **Crzosk's Cam Tool** to identify how many vertexes there are in the triggerzones, so let's open the r100.dat in the tool:



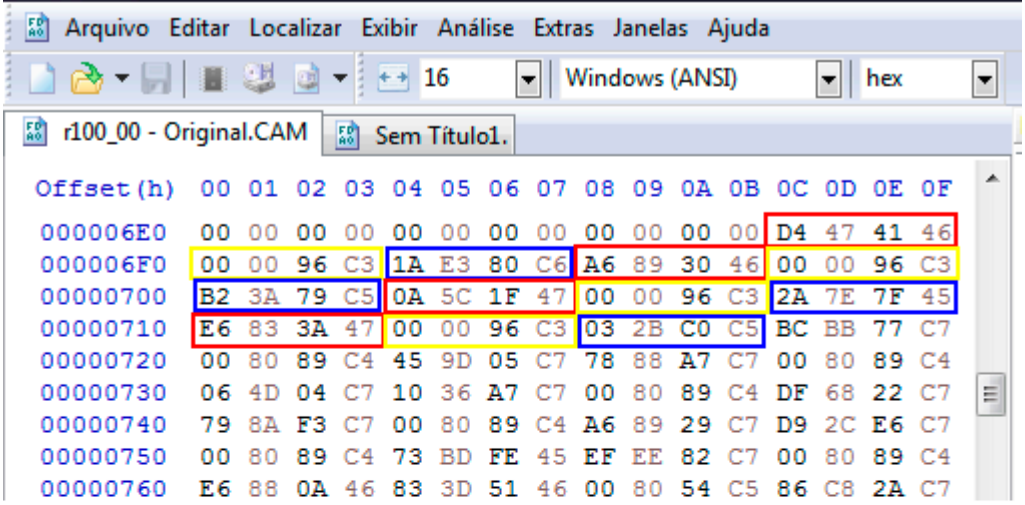
Here you can see the triggerzones vertexes in float values, there are a total of 4 vertexes in this entry, so take a look at the very first one 12,369957.

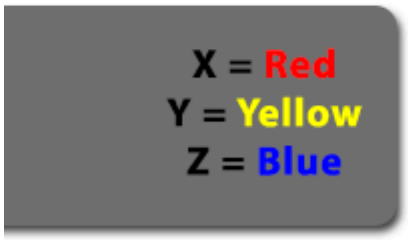
Go back to the HxD and look at the offset **6EC**:



Here you can see, in the float value, that it matches with the value we saw at **Crzosk's tool**, don't worry about this comma in a different place for now.

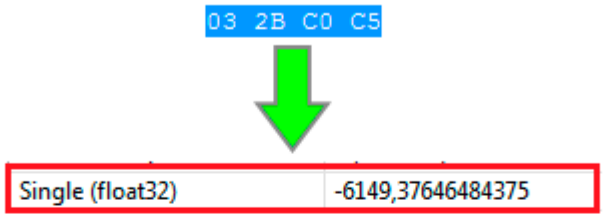
With this information we gathered, now we can obtain all the 4 triggerzone vertexes for one entry, so let's copy all the bytes related to this, they are sequential:

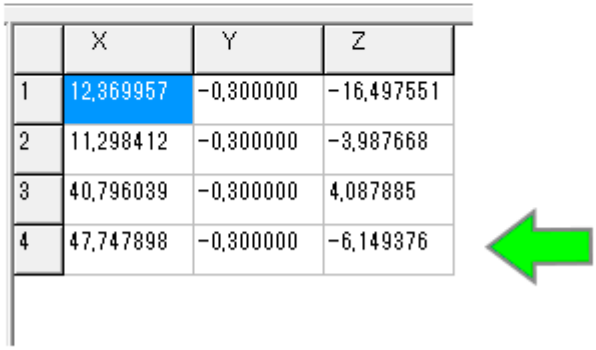




If you are unsure if you selected the correct bytes, always use the Crzosk tool for reference.

Here you can see the last float Z value, and we are correct:





	X	Y	Z
1	12,369957	-0,300000	-16,497551
2	11,298412	-0,300000	-3,987668
3	40,796039	-0,300000	4,087885
4	47,747898	-0,300000	-6,149376

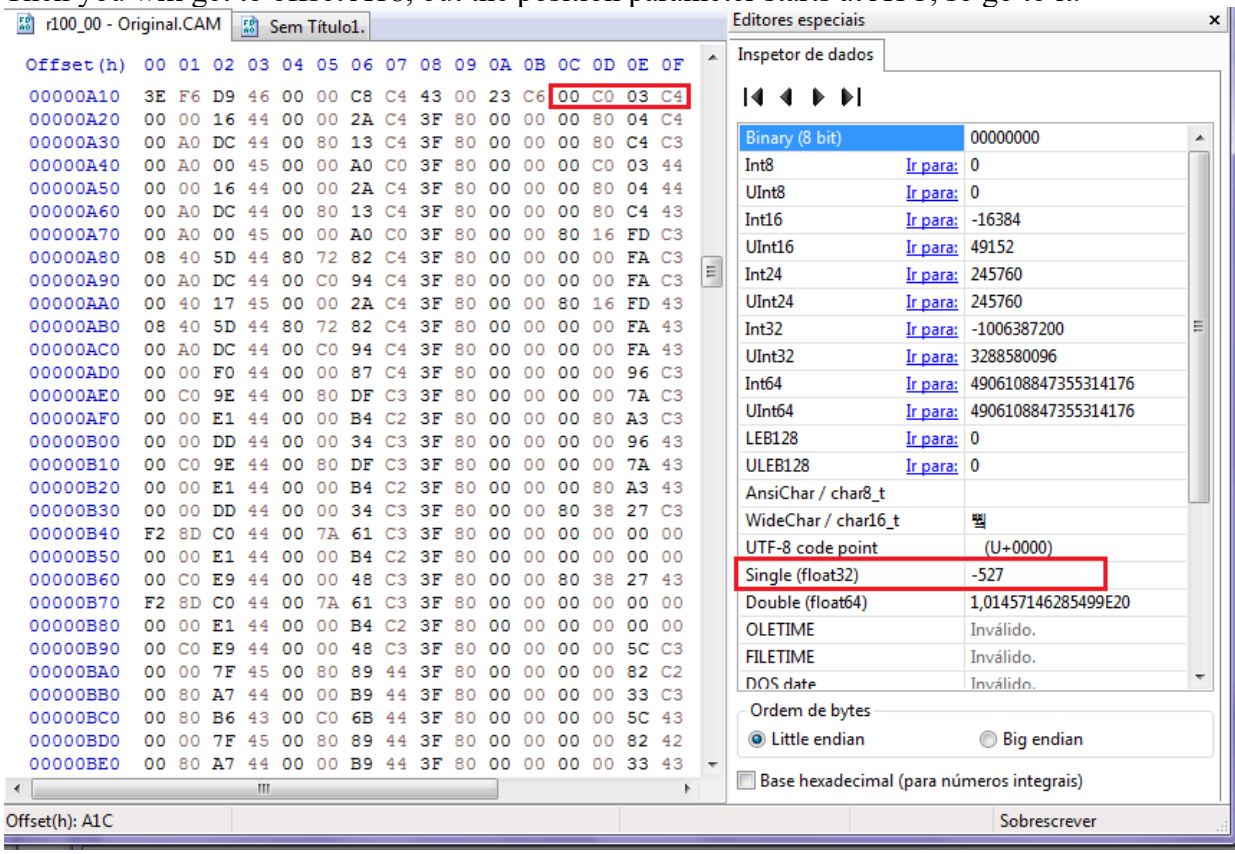
- -

Now we need the last parameters: position, direction and pan zoom.
To find where these parameters starts, we can check the **last triggerzone vertex value** from the **last camera entry**, and right after it will be the first position value.

Search for this float Z value:

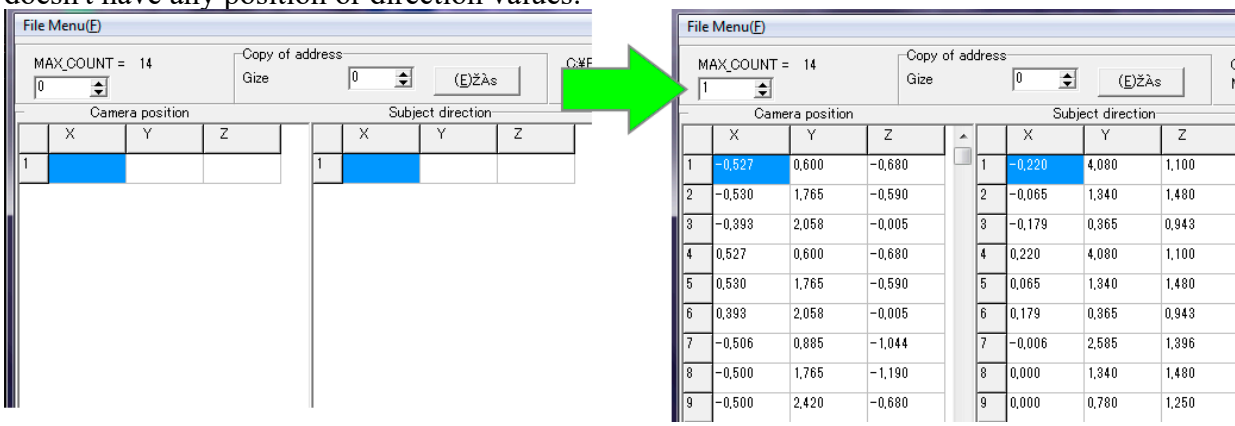
	X	Y	Z
1	29,674211	-1,600000	-6,847506
2	33,258770	-1,600000	-8,622595
3	31,483680	-1,600000	-12,207154
4	27,899121	-1,600000	-10,432065

Then you will get to offset A18, but the position parameter starts at A1C, so go to it.



Here you can see a float value of -527, this is the **first position** of the camera entry.

You can check it out at **Crzosk tool**, but remember to change the index because the first camera doesn't have any position or direction values:



And in this image you can see the very first X position, it is -0,527, which is the same value we saw in HxD, so we are in the correct place.
Position and direction are made of X, Y and Z float values just like triggerzones, but they use another float value as a **separator** between values, which is 3F 80 00 00.

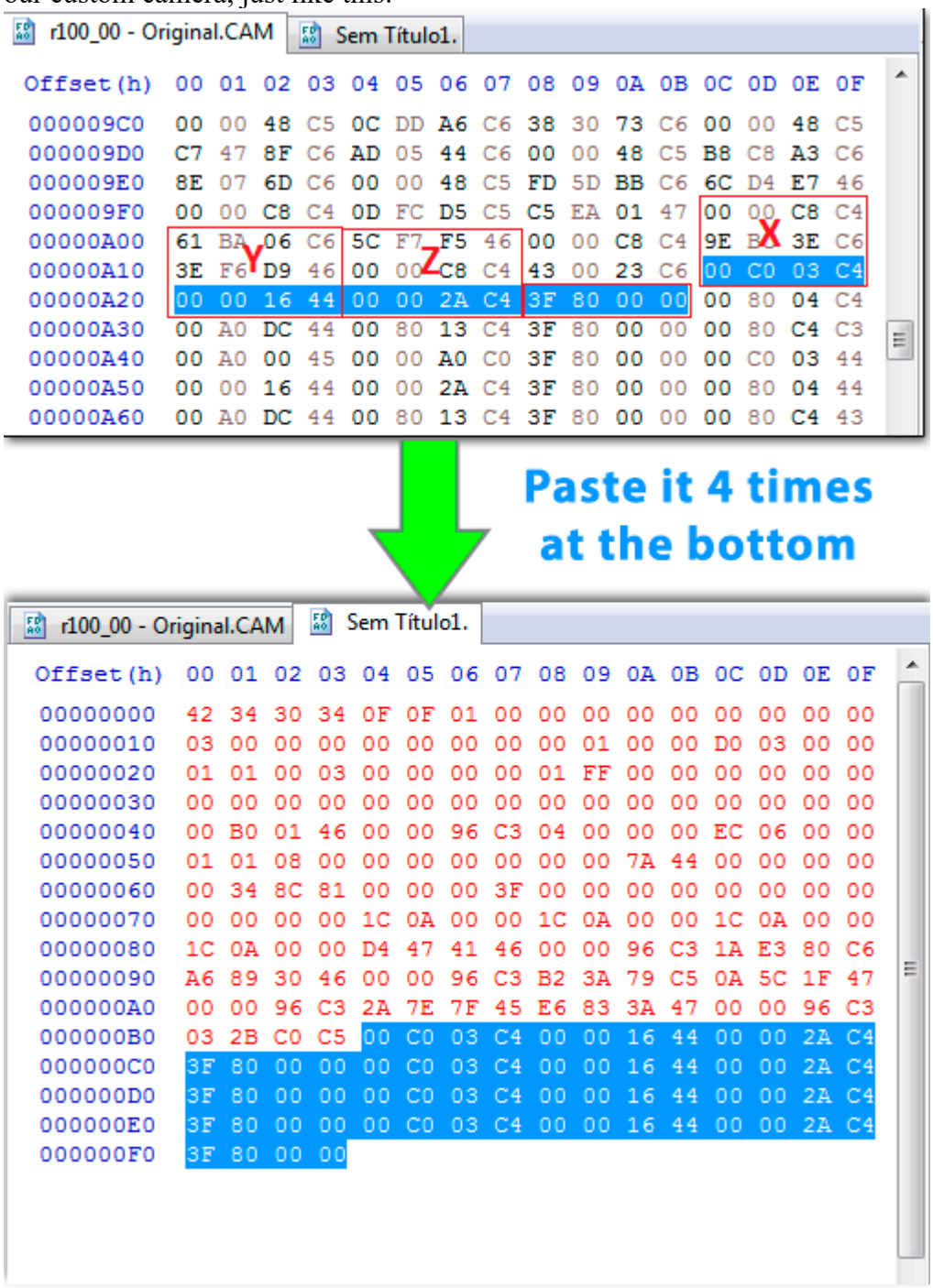
So the data will be like this:

```
00 80 C4 C3 00 A0 00 45 00 00 A0 C0 3F 80 00 00 00 C0 03 44 00 00 16 44 00 00 2A C4
X           Y           Z           SEPARATOR   X           Y           Z
```

Position and direction use the **same amount of bytes**, so that means we can **duplicate** a position to use it as the direction, let's do it now.
On our custom camera we will only need two slots of both position and direction for the data set, so it will look like this:

Pos 1: XXXXYYYYZZZZ 3F 80 00 00
Pos 2: XXXXYYYYZZZZ 3F 80 00 00
Dir 1: XXXXYYYYZZZZ 3F 80 00 00
Dir 2: XXXXYYYYZZZZ 3F 80 00 00

On easier words, we need to copy just one position from r100_00.CAM and paste it 4 times in our custom camera, just like this:

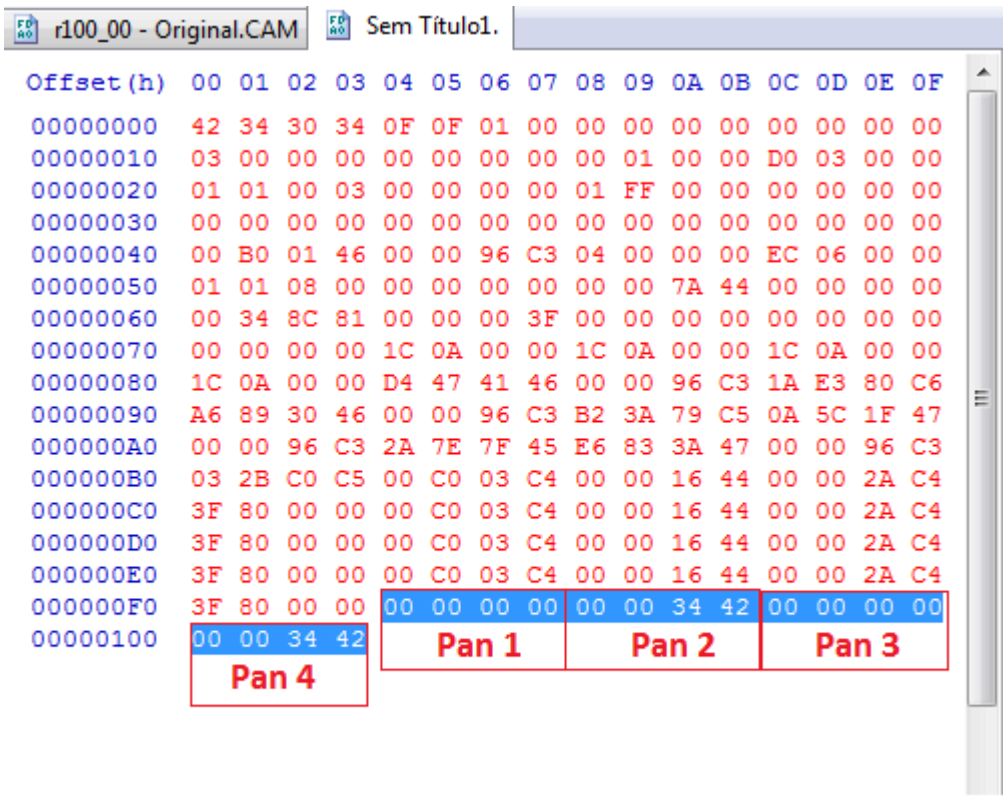


Now we need only the Pan Zoom, finally.
The Pan Zoom is made of two float values for each data set, you can see it on the **Crzosk tool** if you want to check it out.

The first float value makes the camera spin in a creative way, the second float value is responsible for Field of View, my favorite.
Since our camera will have a 2 data set, we will need to get 4 Pan Zoom float values.
(I will explain better about this data set in the next post)

For this, we can search the pan zoom in the r100_00.CAM if we want to, but since we already know how they are made, we can easily insert 4 float values and we're done.

Just like this:



Now just press **CTRL + S** and save it with any name you like, but remember to put .CAM extension at the end, example: mycustomcam.CAM

Done with this part of the tutorial, all the structure have been created and we have successfully created a camera entry from scratch.
However, the camera will not work if you insert it into any room, on the next post I will show you how to fix this and a lot of valuable info.

Last Edit: 29 Jul 2021 at 15:51 by HardRain
TUTORIAL Editing the .CAM file through hex 28 Jul 2021 at 20:46
Post by HardRain on 28 Jul 2021 at 20:46
PART 02

In this part of the tutorial you will learn what some bytes are used for in the cam structure, and you will be introduced to Pointers, the most important part.

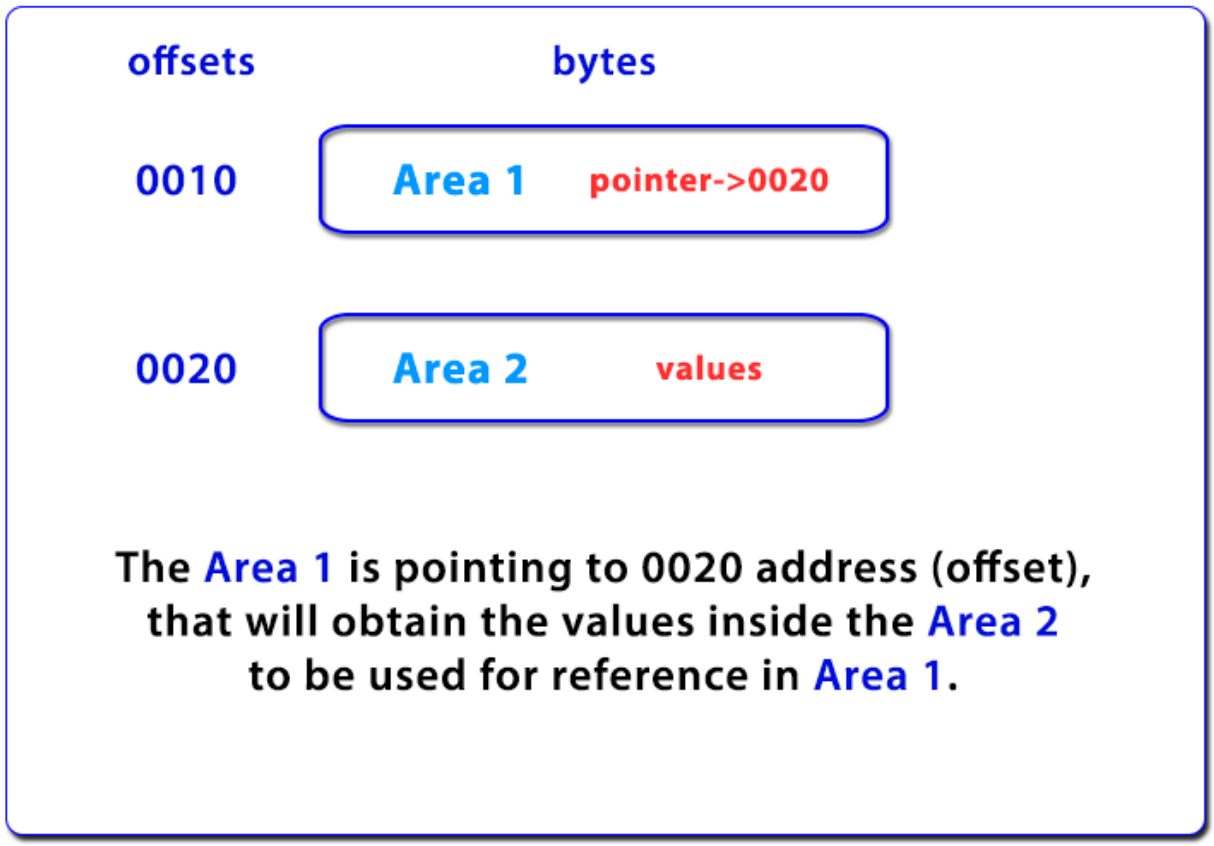
What are Pointers?
Pointers are variables which stores addresses of another memory region, so you can use pointers to get information that is not present in that part of the structure.
Just think about pointers as "arrows" which points to an specific address, in our case address = offset.

HardRain

NEW MEMBER
NEW MEMBER
JUNIOR MEMBER
FULL MEMBER
SENIOR MEMBER

DEDICATED MEMBER
MODDING INFECTED
MODDER
MODERATOR
CFX DESIGNER
ADMIN

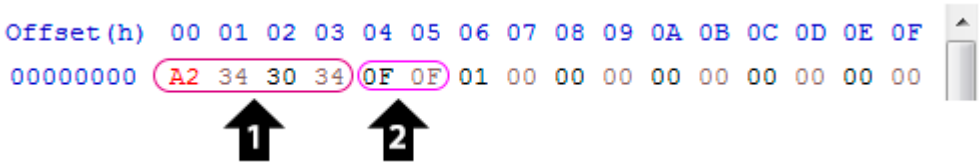
Retired for a while
Posts: 184♂
Member is Online



You will understand better by practicing later, let's move on.

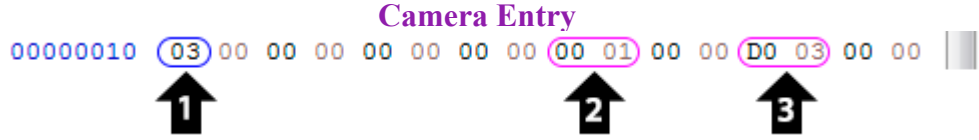
In-Depth details
Now let's take a look at our new custom camera and dissect it to understand what is what.
I am assuming you read the last post to see the cam structure, if don't remember it anymore, just scroll up a bit before continuing.

Header



01: Start of the cam file, without this data the game will not recognize the file.
02: Camera entries index, you will need to update these two values to the total amount of camera entries. In our custom cam we only have 1 entry, so update them both to 01 01.

Camera Entry

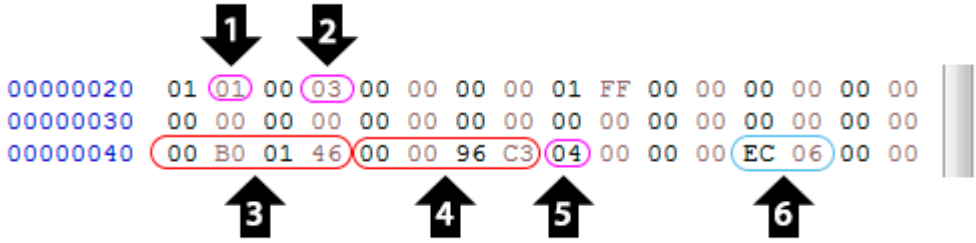


01: Trigger type. The type 03 trigger means that it will be activated as soon as the player enters the triggerzone.
Type 04 triggerzones will be activated when the player opens chests, drawers, cabinets and this sort of things that can be inspected.
There are other types like 23, 63 and some others which I still don't know the real use.

02: Pointer to Camera Properties 1. This pointer tells the file to obtain a value at which offset we put there, as this is the pointer to Camera Properties 1, we will need to update it correctly, so let's do it now.
The cam file is shown to us as **Little Endian** format, but the pointers are **always** in **Big Endian** format. So this means we need to invert the bytes to know the correct offset we need to point at, in our case here we have 00 01 which is equal to 01 00, what means that this pointer is pointing to the data in the 01 00 offset. Which is correct in the original r100_00.CAM, but is wrong in our custom camera, so to fix it we need to put the offset related to the Camera Properties 1 of the first entry, which is 00 20, but you need to update it as **Little Endian**, so put 20 00 and it is done.

03: Pointer to Camera Properties 2. This pointer works the same way as the previous one, you need to edit it and input the offset that starts the Camera Properties 2, which in our custom camera is 00 50, so update it inverted 50 00 and it's done.

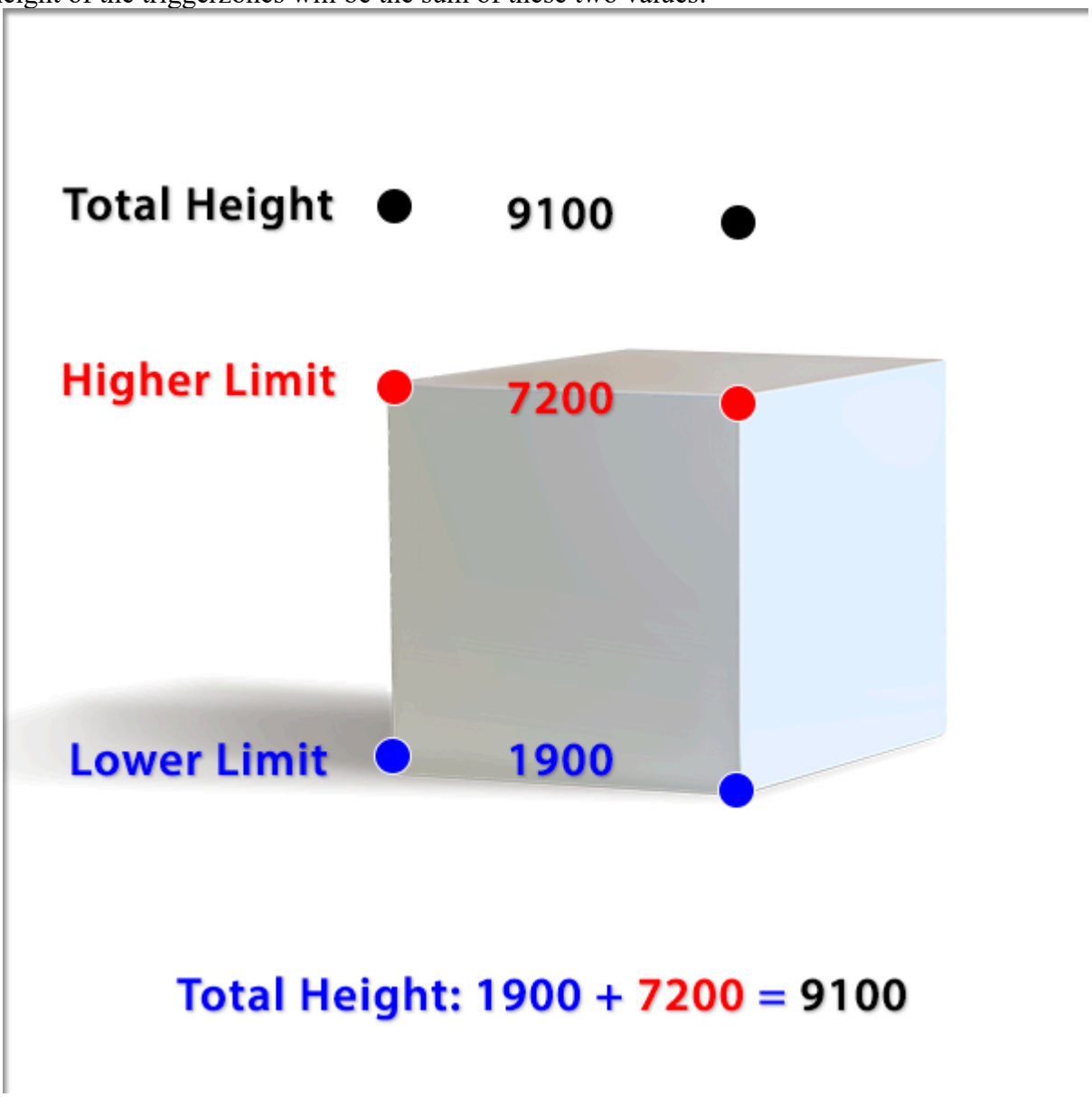
Camera Properties 1



01: LIT Group. This byte activates specified LIT Group present in .LIT file, most used to change ligthing when entering some areas, this is an amazing feature, but I will not cover it up here since Mr.Curious already wrote almost everything you need in his tutorials.

02: Trigger type. You just need to put here the same value you inserted in the first byte of the camera entry, scroll up a little if you forgot.

03 & 04: Higher Limit & Lower Limit. Don't worry about editing these values for now, we can edit them easily in Crzosk's tool.
Higher Limit is where you want the top vertexes to be placed (look at the pic below), this value represents how much taller you want the triggerzones to be.
Lower Limit is the opposite, here you define the lower part of the triggerzones, it is recommended that you put it a little underground to make sure Leon will enter it.
But this is not all. After you input the values, there will be made a sum calculation, so the total height of the triggerzones will be the sum of these two values:

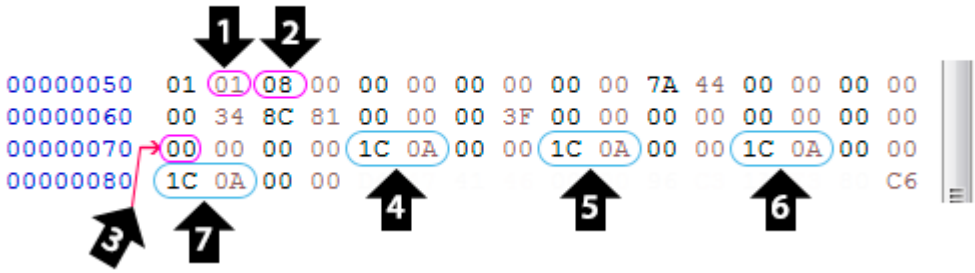


05: Total of triggerzones vertexes. This value defines how much triggerzone vertexes you want to put, and you will need to add correspondent vertexes at triggerzone structure area.

The last arrow is number 06, which is a Pointer pointing to Triggerzones offset. It works the same way as the other pointers, just invert it and replace to the correct offset, which in our custom camera is 00 84, so update it with 84 00 and you're done.
If you're a bit lost about this, check it out in the HxD and look for offset 00000080 in the fourt

column, then look at float value to see the first X triggerzone value.

Cam Properties 2



01: Camera Index. This is updated 1 by 1 at each entry, so if you have two cameras, in the second camera you will need to update this value to 2.

02: Camera type. Now the best part, camera types. There are a total of 9 known different camera types, which I will list here with parts of Mr.Curious descriptions.
0: "Fixed camera in a specified position and direction"
1: "Camera aligns behind player and FOV pulls back as player passes "
2: "Camera position is moved away (over head) & locked, camera does not track the player"
3: "Fixed camera that rotates alongside the player"
4: "Camera position & rotation move to a specific location and pans with player"
5: "360 Camera, moves with right analog stick/camera buttons"
6: "Camera is locked to a specified position & rotation and can be animated"
7: "Camera position & rotation is locked when entering trigger zone"
8: "Over the shoulder camera"

As I have yet not tested all of them, In this tutorial I will be using type 3 for easier understanding. But you can (and should) try all of them to see what happens.
03: Data Set. In the last post I told you about this byte, the Data Set defines how much data will be read. Example, it's 00 now so no data from position, rotation and pan zoom will be read. For type 3 cameras we need 2 data set for it to work, for type 0 cameras we need only 1 data set, but for type 8 cameras we need 18 (24 in decimal) data set for it to work. If you're not understanding what this means, just see how much position and direction data there are in Crzosk's tool.
As we're going to use type 3, just put a 02 there and you're done.

04: Pointer for position. This pointer works like every other, do same steps: invert it, find where the position offset starts (in our custom camera is 00 B4) and put a B4 00 there.

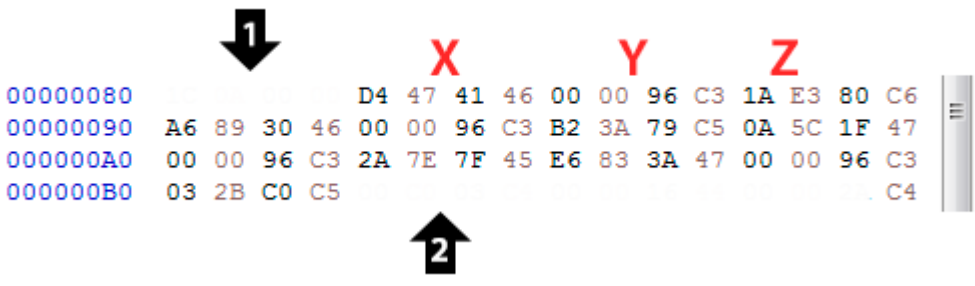
05: Pointer for direction. Same work to do here, find the correct offset that starts the direction and update it. Remeber we are using 2 Data Set, so there will be two positions and after them are two directions. You can see it starts at offset 000000D4, so put a D4 00 there.

06: Pointer for Pan Zoom 1. There are two types of Pan Zoom, one for rotation and other for FOV. And as we are using 2 Data Set, we need two float values for Pan Zoom 1. If you look at our custom camera, you can see it starts at 000000F4, so put a F4 00 there.

07: Pointer for Pan Zoom 2. Works the same way as the previous one, two float values. Starts at offset 000000FC, so put a FC 00 there.

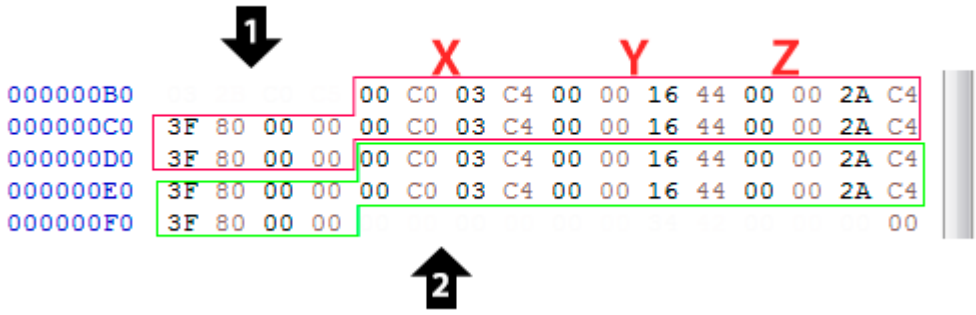
After all of this work, your camera is now ready and can be succesfully opened at Crzosk's tool.

Triggerzones



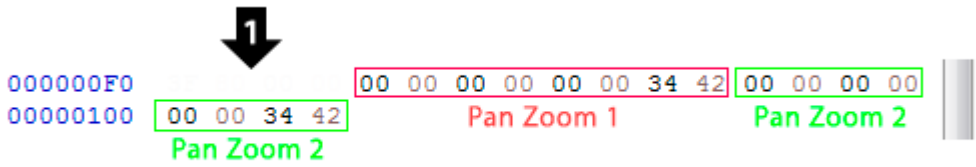
01: End of Cam Properties 2.
02: Beggining of the camera position.
XYZ: Triggerzones. Better explained in previous post.

Position & Direction



01: End of Triggerzones.
02: Beggining of Pan Zoom.
Redish box: Position in X, Y, Z and a separator (better explained in previous post).
Greenish box: Direction in X, Y, Z and a separator.

Pan Zoom



01: End of Direction.
All the rest is self explained and there are more info in the first post.

With this part of the tutorial you learned what is the purpose of some bytes and how they are related to each other, the pointer system can be difficult at the beggining, but keep on practicing until you get used to it.
And remember: you can rename your custom cam to r100_00.CAM and re-import it back at the game to test it.
There is a very nice tutorial over here on how to configure the camera with Crzosk's tool, go ahead and try it out.

Additional Info:
I am aware of some problems while building up entries from scratch because of some optional data that some rooms needs, I am gonna detail them now.

Just before the Triggerzone structure area, there is a 16 bytes row which use is unknown:

r100_00.CAM

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F
00000000	42	34	30	34	02	02	01	00	00	00	00	00	00	00	00	00
00000010	03	00	00	00	00	00	00	00	20	00	00	00	50	00	00	00
00000020	01	01	00	03	00	00	00	00	01	FF	00	00	00	00	00	00
00000030	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
00000040	00	A0	8C	45	00	80	89	C4	06	00	00	00	94	00	00	00
00000050	01	01	03	00	00	00	00	00	00	00	7A	44	00	00	00	00
00000060	00	34	8C	81	00	00	00	3F	00	00	00	00	00	00	00	00
00000070	02	00	00	00	DC	00	00	00	FC	00	00	00	1C	01	00	00
00000080	24	01	00	00	01	01	00	01	00	00	01	00	00	00	00	00
00000090	00	00	00	00	00	04	A6	C7	00	80	89	C4	00	EC	EA	C6
000000A0	00	5F	96	C7	00	80	89	C4	CD	27	EB	C6	00	28	96	C7
000000B0	00	80	89	C4	00	48	DF	C6	80	F1	A5	C7	00	80	89	C4
000000C0	00	A6	DE	C6	00	F2	A5	C7	00	80	89	C4	00	A6	DE	C6
000000D0	00	F2	A5	C7	00	80	89	C4	00	A6	DE	C6	00	5E	AB	C7
000000E0	00	40	4E	45	00	C4	E0	C6	3F	80	00	00	00	5E	AB	C7
000000F0	00	40	4E	45	00	C4	E0	C6	3F	80	00	00	00	00	00	00
00000100	00	00	00	00	00	00	00	00	3F	80	00	00	00	C0	03	C4
00000110	00	00	16	44	00	00	2A	C4	3F	80	00	00	00	00	00	00
00000120	00	00	00	00	00	00	34	42	00	00	34	42	CD	CD	CD	CD
00000130	CD	CD	CD	CD	CD	CD	CD	CD	CD	CD	CD	CD	CD	CD	CD	CD

If your camera does not work, insert these bytes to see if it fixes.

But if the game still crashes or your file doesn't open with Crzosk's tool, try adding 20 bytes of CD just like the pic above. With these fixes, all should work fine.
Last Edit: 30 Jul 2021 at 23:23 by HardRain

HardRain

TUTORIAL Editing the .CAM file through hex 28 Jul 2021 at 20:47
Post by HardRain on 28 Jul 2021 at 20:47
PART 03.1

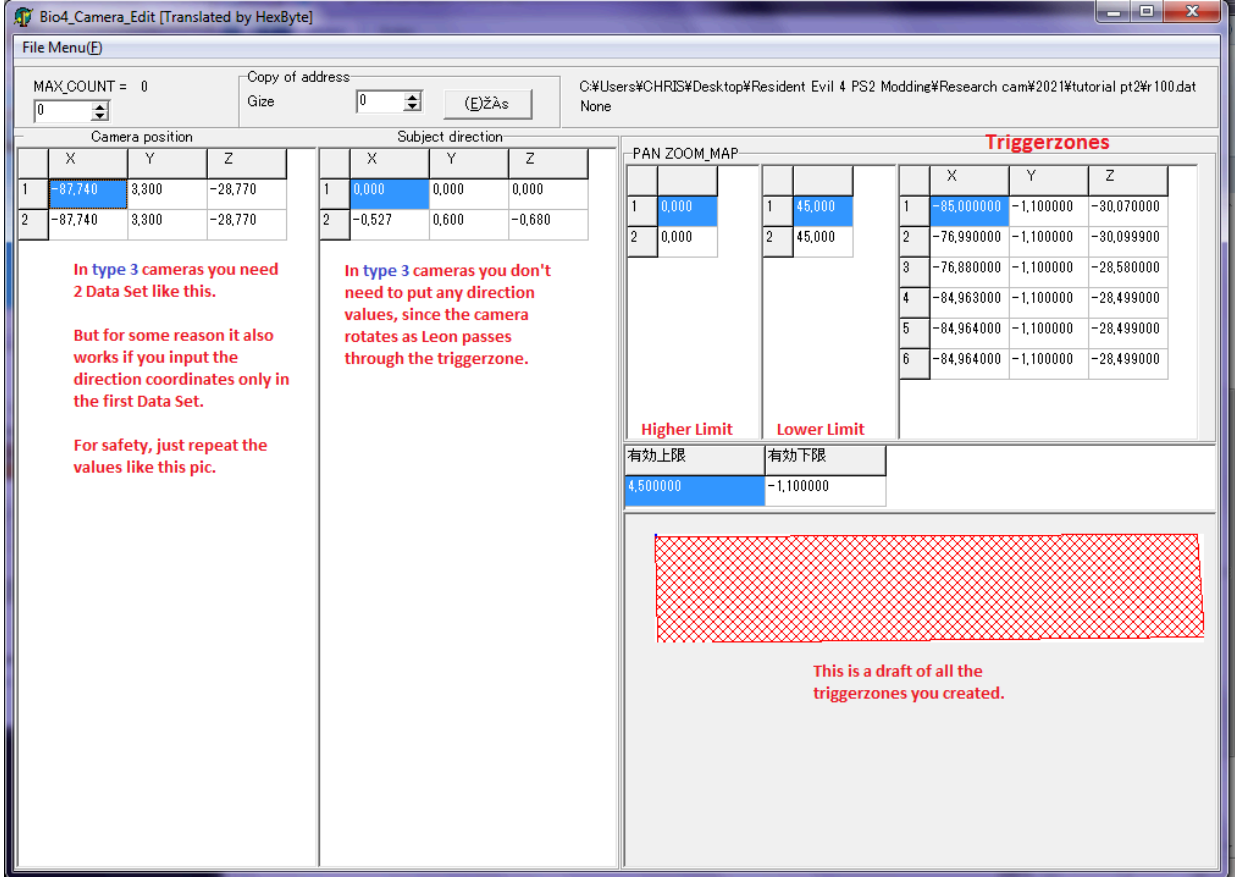
To start this part, I will give you some few tips on how to position the camera, if you already know how to do this, just skip to part **03.2**
We are gonna need to get coordinates to place the camera and build its triggerzone, you can use any tool/trainer you like, but I will show an example using RE4 Level Viewer (Xandreale). You can download it at the first post.
I am assuming you have xfile and xscr files extracted to be able to use Level Viewer with .pmd models, but if you don't know what this is, you can learn over here.

Open up r100_000.pmd with Level Viewer and navigate through the area using arrow keys, and notice there is a coordinate value which changes each time you move the camera, you need those values.

As we know, triggerzones have 4 or 6 vertexes in total to create a **geometric shape** for the player to enter it.
I highly recommend that you start getting the first vertex coordinates from the **left top** side, some rooms seems to have some problems depending on which side you start the triggerzone, so if it does not work, try starting from another side.
So take the coordinates to the area you want the static camera to be, its position and edit them in Crzosk's tool.

Take a look at mine:

Retired for a while
Posts: 184♂
Member is Online



And if you did everything correctly, it should work perfectly:

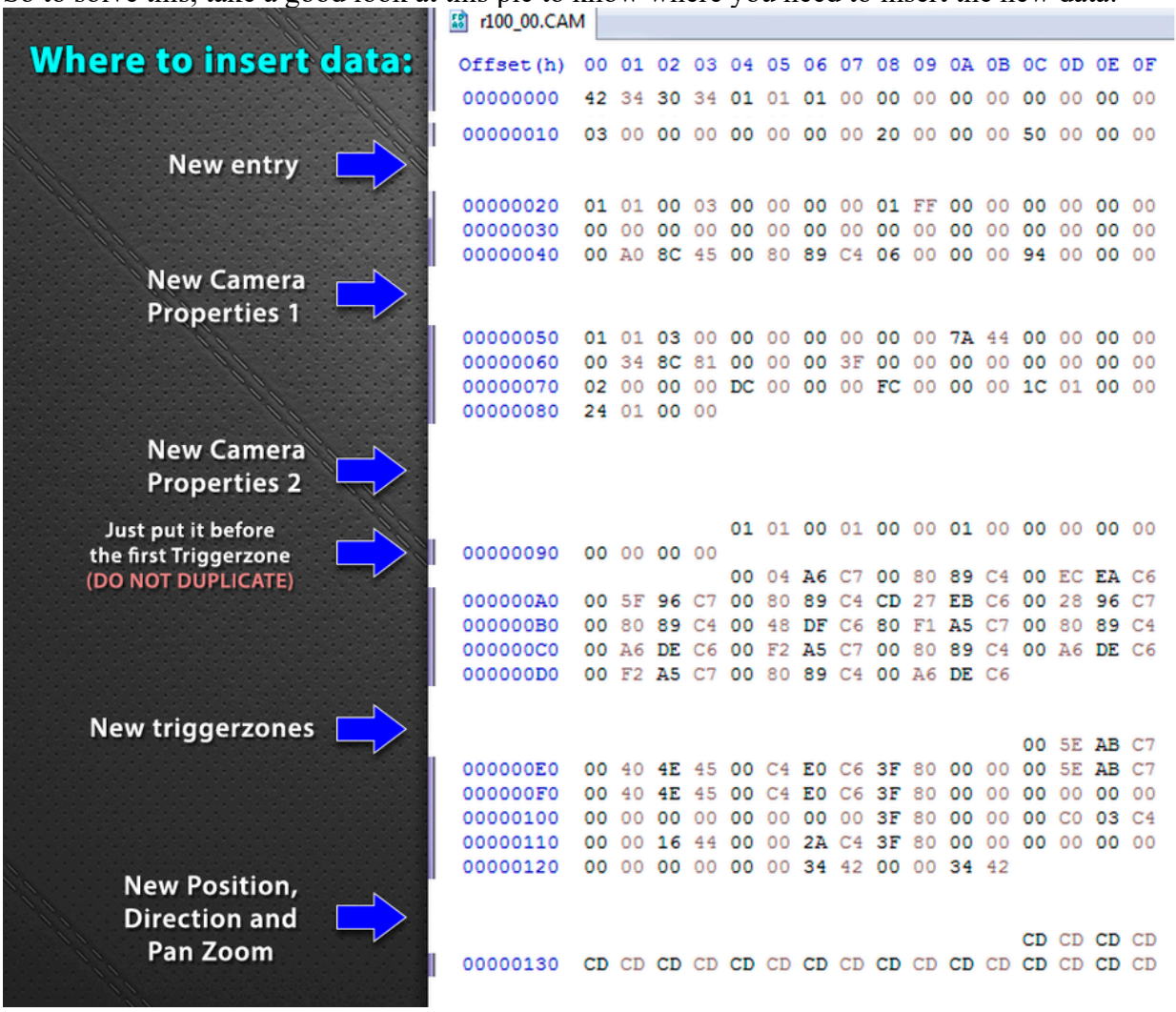
If it doesn't work for you, double check if you edited the height correctly with it trepassing the ground a little, or use mine to compare to yours:

r100_00.CAM

PART 03.2

Done with that part, let's add another camera entry so we can cover more areas with static cameras.
Creating another camera entry is not an super easy task, you can miss something very easily if you don't play close attention, you cannot copy everything and paste it at the final of the file, unfortunately that doesn't work. I wish it would. Really.

So to solve this, take a good look at this pic to know where you need to insert the new data:

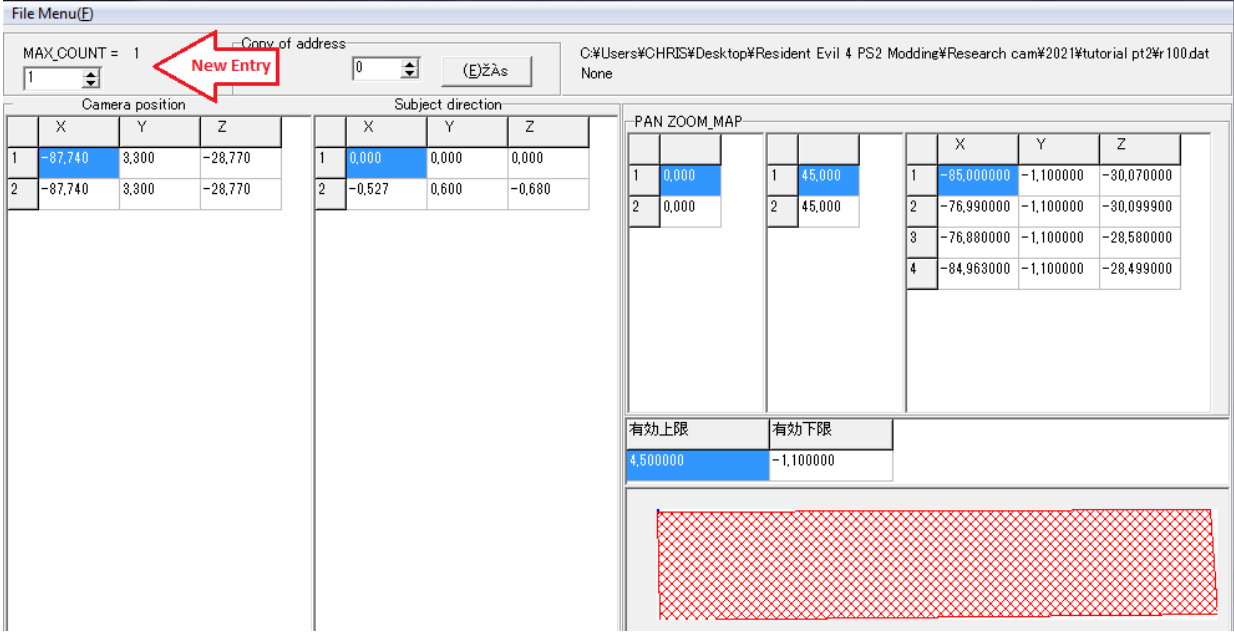


In a few colored words:

Entry 1
Entry 2
Cam Prop 1
Cam Prop 1
Cam Prop 2
Cam Prop 2
Unknown data
Triggerzones first entry
Triggerzones second entry
Pos, Dir, Pan Zoom first entry
Pos, Dir, Pan Zoom second entry

Knowing about this structure, just duplicate every part of it and insert in the correct locations.
Remember to **update the indexes from the header and the index for the second Camera Properties 2**, this was explained previously.
The worst part is that you must update **ALL THE POINTERS** each time you add a new camera entry, so if you do not remember how to locate them, scroll up a little to find the explanation.

If you did everything correctly, repack the files and open in Crzosk's tool again, then you will see this new entry available:



I choose to use just 4 triggerzones for this entry, that is an optional step.

Now just edit this new entry, search for the desirable coordinates you want.
(One tip: try to train by creating cameras for every corner of the first house)

You can also have my edited camera with two entries here, in case you want to compare:
r100_00.CAM 2 entries

Done with this part, let's go to the last one.
Last Edit: 3 Aug 2021 at 22:48 by HardRain

HardRain

TUTORIAL Editing the .CAM file through hex 28 Jul 2021 at 20:47
Post by HardRain on 28 Jul 2021 at 20:47
PART 04

Ok, for the last part we want to insert our custom camera entry into the original cam file, r100_00.CAM.
As this is not very hard, it is very tedious. Especially if you're going to add multiple entries. But let's move.

First of all, open your original r100_00.CAM file and your custom camera on HxD.
Take a look at the header for its indexes values, it is 0F 0F, but if we are going to add a new camera, we need to update it to 10 10. (0F = 15, 10 = 16).
In our second move we need to copy our custom camera entry structure part and paste it below all the entries in the original cam:

ADMIN

Retired for a while
Posts: 184♂
Member is Online

r100_00 - Original.CAM | mycustomcam.CAM

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F
00000000	42	34	30	34	02	02	01	00	00	00	00	00	00	00	00	00
00000010	03	00	00	00	00	00	00	00	20	00	00	00	50	00	00	00
00000020	01	01	00	03	00	00	00	00	01	FF	00	00	00	00	00	00
00000030	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
00000040	00	A0	8C	45	00	80	89	C4	06				00	94	00	00
00000050	01	01	03	00	00	00	00	00	00				44	00	00	00
00000060	00	34	8C	81	00	00	00	3F	00				00	00	00	00
00000070	02	00	00	00	DC	00	00	00	FC				00	1C	01	00

r100_00 - Original.CAM | mycustomcam.CAM

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F
00000000	42	34	30	34	10	10	01	00	00	00	00	00	00	00	00	00
00000010	03	00	00	00	00	00	00	00	00	01	00	00	D0	03	00	00
00000020	03	00	00	00	00	00	00	00	30	01	00	00	04	04	00	00
00000030	03	00	00	00	00	00	00	00	60	01	00	00	38	04	00	00
00000040	03	00	00	00	00	00	00	00	90	01	00	00	6C	04	00	00
00000050	23	00	00	00	00	00	00	00	C0	01	00	00	A0	04	00	00
00000060	23	00	00	00	00	00	00	00	F0	01	00	00	D4	04	00	00
00000070	04	00	00	00	00	00	00	00	20	02	00	00	08	05	00	00
00000080	04	00	00	00	00	00	00	00	50	02	00	00	3C	05	00	00
00000090	04	00	00	00	00	00	00	00	80	02	00	00	70	05	00	00
000000A0	04	00	00	00	00	00	00	00	B0	02	00	00	A4	05	00	00
000000B0	04	00	00	00	00	00	00	00	E0	02	00	00	D8	05	00	00
000000C0	03	00	00	00	00	00	00	00	10	03	00	00	0C	06	00	00
000000D0	04	00	00	00	00	00	00	00	40	03	00	00	40	06	00	00
000000E0	04	00	00	00	00	00	00	00	70	03	00	00	74	06	00	00
000000F0	04	00	00	00	00	00	00	00	A0	03	00	00	A8	06	00	00
00000100	03	00	00	00	00	00	00	00	20	00	00	00	50	00	00	00
00000110	01	01	00	03	00	00	00	00	01	FF	00	00	00	00	00	00
00000120	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
00000130	00	B0	01	46	00	00	96	C3	04	00	00	00	EC	06	00	00

Now we need to paste our custom Camera Properties 1 below all Camera Properties 1 in the original cam. By following all the previous posts you should know where it starts, but if you're lost, you can follow the LIT group byte, it will increase from 01 to 0F.

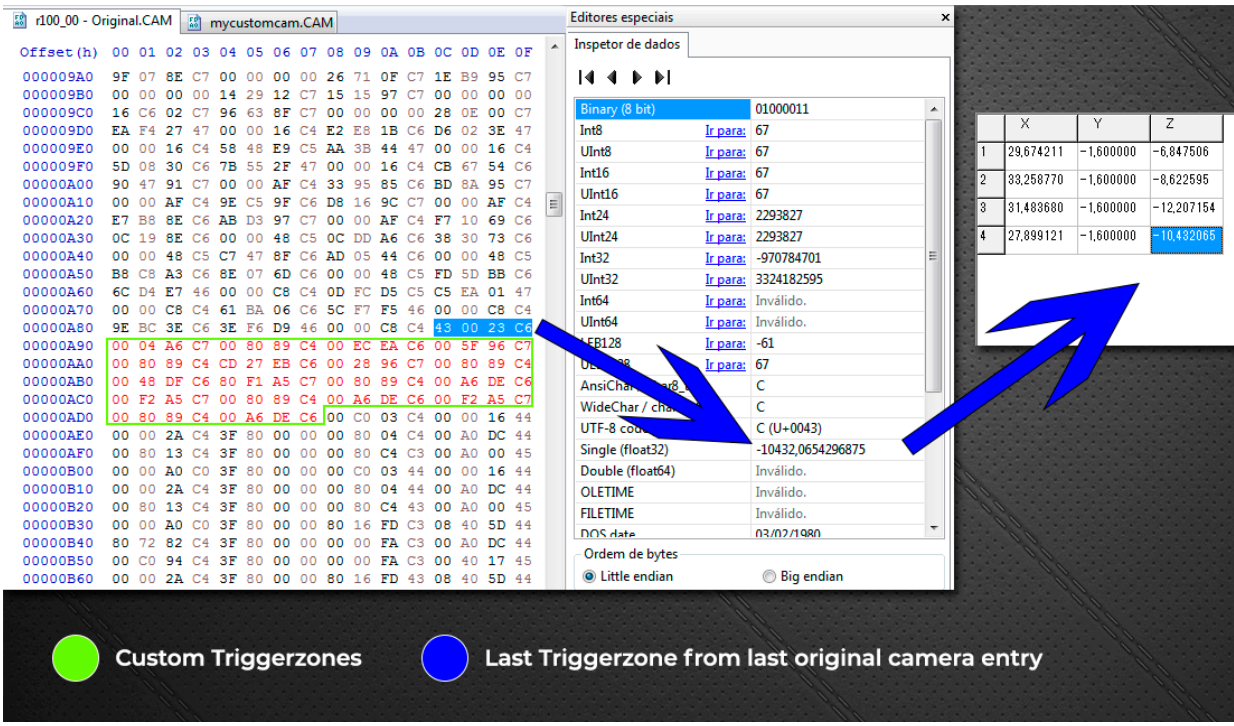
After pasting Camera Properties 1, we need to do the same with Camera Properties 2. But remember to update the index in this part, and remember this structure is made of **34 bytes**. Paste your custom Camera Properties 2 just before that row of unknown bytes:
01 01 00 01 00 00 00 00 00 00 00 00 00 00 00 00

r100_00 - Original.CAM | mycustomcam.CAM

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F
00000680	01	0D	06	00	00	00	00	00	00	00	7A	44	00	00	00	00
00000690	30	23	00	00	00	00	00	00	00	00	00	00	00	00	00	00
000006A0	02	00	00	00	2C	22	00	00	4C	22	00	00	6C	22	00	00
000006B0	74	22	00	00	01	0E	06	00	00	00	00	00	00	00	7A	44
000006C0	00	00	00	00	34	23	00	00	00	00	00	00	00	00	00	00
000006D0	00	00	00	00	02	00	00	00	7C	22	00	00	9C	22	00	00
000006E0	BC	22	00	00	C4	22	00	00	01	0F	06	00	00	00	00	00
000006F0	00	00	7A	44	00	00	00	00	38	23	00	00	00	00	00	00
00000700	00	00	00	00	00	00	00	00	02	00	00	00	CC	22	00	00
00000710	EC	22	00	00	0C	23	00	00	14	23	00	00	01	10	03	00
00000720	00	00	00	00	00	00	7A	44	00	00	00	00	00	34	8C	81
00000730	00	00	00	3F	00	00	00	00	00	00	00	00	02	00	00	00
00000740	DC	00	00	00	FC	00	00	00	1C	01	00	00	24	01	00	00
00000750	01	01	00	01	00	00	00	00	00	00	00	00	00	00	00	00
00000760	D4	47	41	46	00	00	96	C3	1A	E3	80	C6	A6	89	30	46
00000770	00	00	96	C3	B2	3A	79	C5	0A	5C	1F	47	00	00	96	C3
00000780	2A	7E	7F	45	E6	83	3A	47	00	00	96	C3	03	2B	C0	C5
00000790	BC	BB	77	C7	00	80	89	C4	45	9D	05	C7	78	88	A7	C7

Last Cam Prop Custom Cam Prop Unknown data

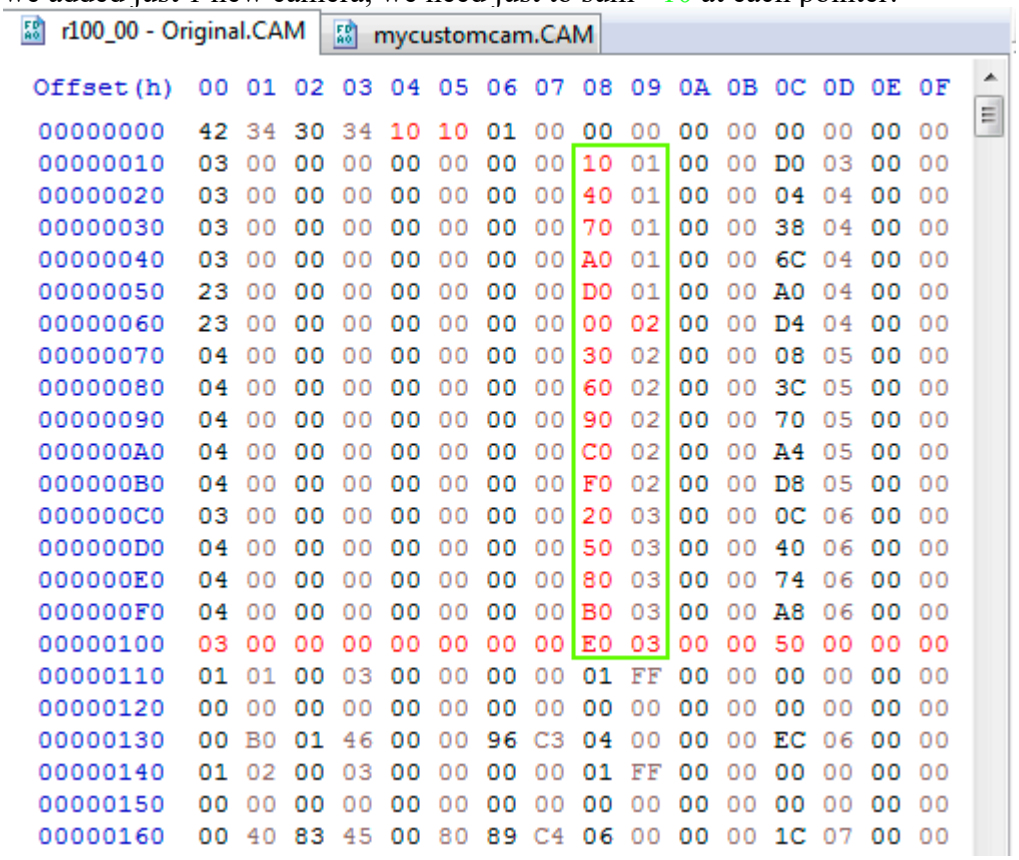
Now you need to paste your custom triggerzones below all original triggerzones.
To be sure you are in the correct offset, you can always check the **float value** and check if it is the same at Crzosk's tool.



Now we just need to paste our custom Position, Direction and Pan Zoom at the end of the file, before any CD CD CD CD.
After all pasting is done, yeah, we need to update **ALL THE POINTERS** again.
Pointers from camera entry, cam prop 1 & 2.

You need to be very cautious while doing this, as this is what makes the cameras work, so everything needs to be pointing at the correct place.
I will give some tips here.

Look at your original cam entries pointers, you can see they update in a pattern:
00 01, 30 01, 60 01, 90 01, C0 01, F0 01, 20 02 ... They update after each 30 bytes (in hexadecimal values).
So you can use this same method to update these pointers and include your new entry, and as we added just 1 new camera, we need just to sum +10 at each pointer:



Now when we are updating the next pointers, there is also a way to do it "faster".
If you take a look at **Int16** value at each pointer, you can see all of them increases by 52. So you can open up your calculator and use this as an advantage. Update all of them.

When you're done, the last pointer will be pointing at the offset of your custom Camera Properties 2:

r100_00 - Original.CAM | mycustomcam.CAM

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F
00000000	42	34	30	34	10	10	01	00	00	00	00	00	00	00	00	00
00000010	03	00	00	00	00	00	00	00	10	01	00	00	10	04	00	00
00000020	03	00	00	00	00	00	00	00	40	01	00	00	44	04	00	00
00000030	03	00	00	00	00	00	00	00	70	01	00	00	78	04	00	00
00000040	03	00	00	00	00	00	00	00	A0	01	00	00	AC	04	00	00
00000050	23	00	00	00	00	00	00	00	D0	01	00	00	E0	04	00	00
00000060	23	00	00	00	00	00	00	00	00	02	00	00	14	05	00	00
00000070	04	00	00	00	00	00	00	00	30	02	00	00	48	05	00	00
00000080	04	00	00	00	00	00	00	00	60	02	00	00	7C	05	00	00
00000090	04	00	00	00	00	00	00	00	90	02	00	00	B0	05	00	00
000000A0	04	00	00	00	00	00	00	00	C0	02	00	00	E4	05	00	00
000000B0	04	00	00	00	00	00	00	00	F0	02	00	00	18	06	00	00
000000C0	03	00	00	00	00	00	00	00	20	03	00	00	4C	06	00	00
000000D0	04	00	00	00	00	00	00	00	50	03	00	00	80	06	00	00
000000E0	04	00	00	00	00	00	00	00	80	03	00	00	B4	06	00	00
000000F0	04	00	00	00	00	00	00	00	B0	03	00	00	E8	06	00	00
00000100	03	00	00	00	00	00	00	00	E0	03	00	00	1C	07	00	00
00000110	01	01	00	03	00	00	00	00	01	FF	00	00	00	00	00	00
00000120	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
00000130	00	B0	01	46	00	00	96	C3	04	00	00	00	EC	06	00	00
00000140	01	02	00	03	00	00	00	00	01	FF	00	00	00	00	00	00
00000150	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00

Here comes the next part where we need to update all the pointers from the Camera Properties 1. Remember they point to the triggerzones, so you may want to use Crzosk's tool to see if you are in the correct offset.

I have yet not found an easy way to update them, since the math here depends on how much triggerzones vertexes are defined.
Each time it's a 4 vertexes triggerzone, it increases by 48 at Int16 value.
Each time it's a 6 vertexes triggerzone, it increases by 72 at Int16 value.

With this information, you can take a good look at the **Vertexes byte** and see if there is a 4 or 6 there, then you can use the calculator again and do the math.
The second alternative is to check the triggerzones values at **float value** and compare at Crzosk's tool.

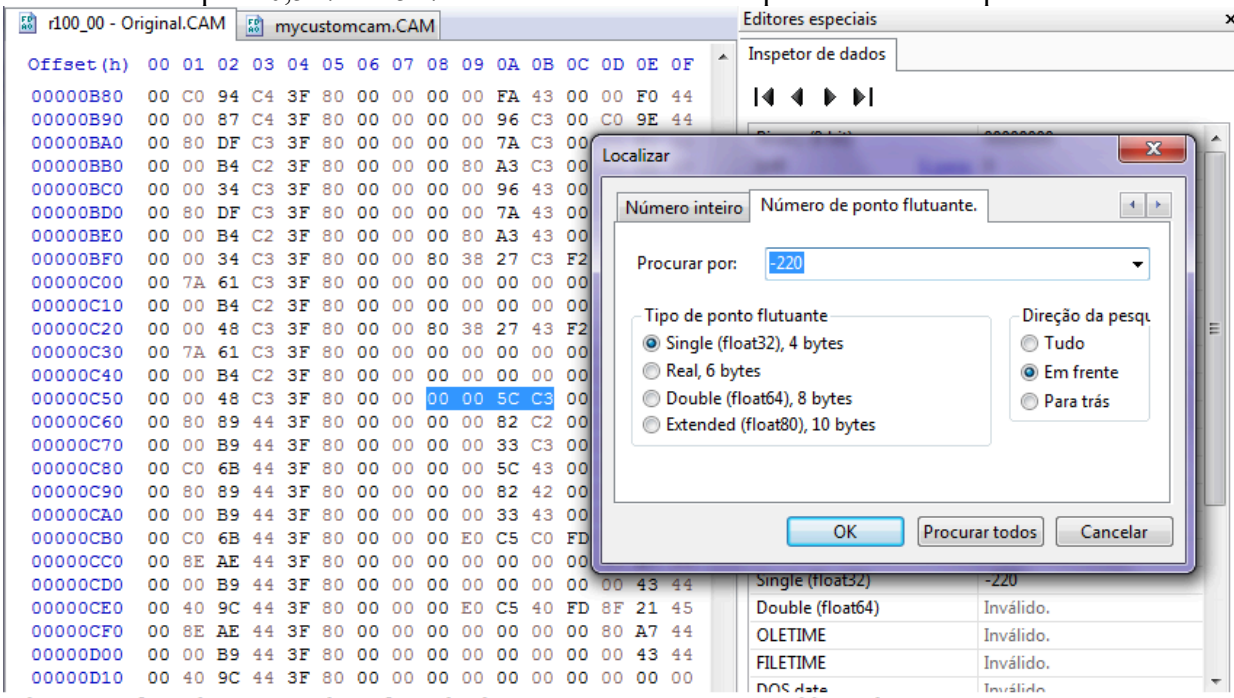
If you did everything correctly, you can see the last pointer pointing at your custom triggerzones:

r100_00 - Original.CAM | mycustomcam.CAM

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F
00000310	00	00	7A	44	00	00	00	00	04	00	00	00	A0	09	00	00
00000320	01	0C	00	03	00	00	00	00	01	FF	00	00	00	00	00	00
00000330	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
00000340	00	00	16	45	00	00	16	C4	04	00	00	00	D0	09	00	00
00000350	01	0D	00	04	00	00	00	00	01	FF	00	00	00	00	00	00
00000360	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
00000370	00	00	7A	44	00	00	AF	C4	04	00	00	00	00	0A	00	00
00000380	01	0E	00	04	00	00	00	00	01	FF	00	00	00	00	00	00
00000390	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
000003A0	00	00	7A	44	00	00	48	C5	04	00	00	00	30	0A	00	00
000003B0	01	0F	00	04	00	00	00	00	01	FF	00	00	00	00	00	00
000003C0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
000003D0	00	00	7A	44	00	00	C8	C4	04	00	00	00	60	0A	00	00
000003E0	01	01	00	03	00	00	00	00	01	FF	00	00	00	00	00	00
000003F0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
00000400	00	A0	8C	45	00	80	89	C4	06	00	00	00	90	0A	00	00
00000410	01	01	08	00	00	00	00	00	00	00	7A	44	00	00	00	00
00000420	00	34	8C	81	00	00	00	3F	00	00	00	00	00	00	00	00
00000430	00	00	00	00	1C	0A	00	00	1C	0A	00	00	1C	0A	00	00
00000440	1C	0A	00	00	01	02	08	00	00	00	00	00	00	00	7A	44
00000450	00	00	00	00	00	34	8C	81	00	00	00	00	00	00	00	00
00000460	00	00	00	00	18	00	00	00	1C	0A	00	00	9C	0B	00	00

Okay, now with this last and worst part, we need to update all the pointers to position, direction and pan zoom.
The only method I found to this is look at Crzosk's tool everytime to see if we are in the correct offset, so unless you find a better way, we must do this.

You can also search for the values that are in the cam tool, just remember to remove the zero before it. Example: -0,527 -> -527 and search in HxD with pointer number option.



This is our first direction value, if you look at camera entry 1 in Crzosk's tool.
So you can search for every first and last position and direction value to find them quickly.

Update all the pointers correctly, then your .cam file will be ready to go.
Be patient and attentive.

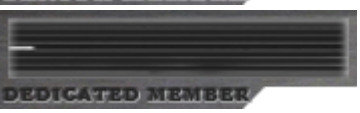
And this concludes the tutorial, I hope you could understand every step of it and you learned something from it.
If you have any questions, please just ask here.

Stay cool.
Last Edit: 3 Aug 2021 at 22:47 by HardRain

Biohazard4X
TUTORIAL Editing the .CAM file through hex 29 Jul 2021 at 18:52 HardRain likes this
Post by Biohazard4X on 29 Jul 2021 at 18:52

So far so good! Good work for decoding a file completely in hex with some small tools. Defeinetly alot of work and attention in these guides, more than i would ever stuck in.

Good Work!



080
Posts: 383

Quick Reply

Post Quick Reply Spell Check

Shoutbox

WELCOME TO RE MODDING. Please use [u-rl][/url] (remove the - in the first tag) tags at the start/end of links.
frozenburnside: Ever wanted to remove the simple motion blur on the PS2 port of Code Veronica? Now you can, and also turn off the fog as well. 21 Aug 2026 at 15:40
frozenburnside: www.youtube.com/watch?v=6_xxnhz2uzE 21 Aug 2026 at 15:40
danielegamer: how do I exchange itens in merc RE5? 23 Aug 2026 at 17:36
danielegamer: I already got mercenaries editor but I still can't find the item folder to edit 23 Aug 2026 at 17:53
Majin-Vegeta: www.youtube.com/watch?v=3-2QrSGLaCI 23 Aug 2026 at 19:31
Majin-Vegeta: Episode 20/32 ✅ | What did you think of the changes to this scenario? Leave your feedback below! And if you liked the mod, subscribe and leave a like so you don't miss the next maps! 🔥 23 Aug 2026 at 19:31
fykefiq: 평소 일정이 바빠 관리받을 시간을 따로 내기 어려웠는데 원하는 장소에서 받을 수 있어 정말 편했어요. 출장마사지gogomsg.clickn.co.kr/ 는 이동 시간을 줄일 수 있다는 장점이 있고, 목과 어깨처럼 쉽게 뭉치는 부분을 집중적으로 관리해 바쁜 일상 속 휴식 시간을 알차게 활용할 수 있었습니다. 25 Aug 2026 at 14:54
MaxMotors: buongiorno a tutti ragazzi. sono nuovo del sito. dove trovo le mod grafiche di resident evil 2, 3, 4 remake? qualcuno puo aiutarmi, e magari scrivendomi il link in privato? grazie mille 26 Aug 2026 at 06:45
Majin-Vegeta: www.youtube.com/watch?v=zMvfnnYLA7o 26 Aug 2026 at 19:50
Majin-Vegeta: Episode 21/32 ✅ | What did you think of the changes to this scenario? Leave your feedback below! And if you liked the mod, subscribe and leave a like so you don't miss the next maps! 🔥 26 Aug 2026 at 19:50
frozenburnside: I think with a bit less fog and no motion blur, CVX looks really good on PS2: www.youtube.com/watch?v=ek2qQeq3n3g&t=20s 28 Aug 2026 at 18:47
Majin-Vegeta: www.youtube.com/watch?v=W5zA66bvTAc 28 Aug 2026 at 19:08
Majin-Vegeta: Episode 22/32 ✅ | What did you think of the changes to this scenario? Leave your feedback below! And if you liked the mod, subscribe and leave a like so you don't miss the next maps! 🔥 28 Aug 2026 at 19:08
Majin-Vegeta: www.youtube.com/watch?v=WPb6u3dTX_0 30 Aug 2026 at 18:41
Majin-Vegeta: Episode 23/32 ✅ | What did you think of the changes to this scenario? Leave your feedback below! And if you liked the mod, subscribe and leave a like so you don't miss the next maps! 🔥 30 Aug 2026 at 18:41
Majin-Vegeta: www.youtube.com/watch?v=9AZARpJUYko 1 Sep 2026 at 18:45
Majin-Vegeta: Episode 24/32 ✅ | What did you think of the changes to this scenario? Leave your feedback below! And if you liked the mod, subscribe and leave a like so you don't miss the next maps! 🔥 1 Sep 2026 at 18:45
Majin-Vegeta: www.youtube.com/watch?v=XgEahgKBJ0E 3 Sep 2026 at 19:16

Majin-Vegeta: Episode 25/32 ✅ | What did you think of the changes to this scenario? Leave your feedback below! And if you liked the mod, subscribe and leave a like so you don't miss the next maps! 🔥 3 Sep 2026 at 19:16
residentevil420: can somebody mod laura from silent hill over sherry yesterday at 09:54